

0507_STUDY_이재호

rcipy.task.Future

`future` 는 ROS 2 Python 클라이언트 라이브러리인 `rcipy` 에서 제공하는 `rcipy.task.Future` 클래스의 인스턴스입니다. 구조를 살펴보면, 기본적으로 **비동기 작업의 상태와 결과를 저장하는 클래스**이며, 주요 속성과 메서드는 다음과 같습니다.

속성명	설명
<code>done_</code>	작업 완료 여부 (<code>True/False</code>)
<code>_result</code>	완료된 경우의 결과값 (<code>response</code>)
<code>_callbacks</code>	완료 시 호출할 콜백 리스트

메서드명	설명
<code>done()</code>	작업이 완료되었는지 확인 (<code>bool</code> 반환)
<code>result()</code>	완료된 경우 결과 반환
<code>add_done_callback(fn)</code>	작업 완료 시 실행할 함수 등록
<code>set_result(value)</code>	내부적으로 호출되어 결과 저장
<code>set_exception(exc)</code>	예외 발생 시 설정

```
# 1. 요청 보냄 → future 객체 생성
future = self.cli.call_async(self.req)

# 2. 이벤트 루프가 돌면서 응답을 받으면
#   내부적으로 set_result(response) 호출됨

# 3. 이후 사용자가 다음과 같이 결과를 확인함
if future.done():
    response = future.result()
```

Action에서의 Future

Action 클라이언트의 Sending Goal(Req)

- Action server를 생성할 때 callback function을 설정하면 Action server가 goal을 받을 때 callback function을 실행 (feedback → result 처리, return result)
- 이때, callback function으로 rclpy.action.server.ServerGoalHandle 클래스의 인스턴스가 인수로 입력
- 이때, callback function으로 rclpy.action.server.ServerGoalHandle 클래스의 인스턴스가 인수로 입력
 - Attributes and Methods

속성	설명
<code>.goal</code>	클라이언트가 보낸 goal 메시지 객체
<code>.goal_id</code>	고유한 goal ID (<code>UUID</code>)
<code>.status</code>	현재 goal 상태 (예: 수락됨, 취소됨, 성공 등)
<code>.handle</code>	내부의 rclpy의 C 확장 핸들
<code>.goal_info</code>	내부적으로 goal에 대한 메타 정보 (goal_id 등)

메서드	반환값	설명
<code>get_goal()</code>	goal 메시지	클라이언트가 보낸 goal 객체 반환
<code>publish_feedback(feedback_msg)</code>	None	중간 피드백을 클라이언트에게 전송
<code>is_cancel_requested()</code>	bool	클라이언트가 goal 취소를 요청했는지 확인
<code>canceled()</code>	None	goal을 "취소됨" 상태로 마무리하고 결과 반환
<code>succeed()</code>	None	goal을 "성공" 상태로 마무리
<code>abort()</code>	None	goal을 "실패" 상태로 마무리

함수명	역할	<code>Future.result()</code> 의 반환 타입	반환 구조 설명
<code>call_async()</code>	서비스 요청	서비스 응답 메시지 객체	사용한 <code>.srv</code> 의 <code>Response</code> 타입

<code>send_goal_async()</code>	Action goal 전송	<code>ClientGoalHandle</code> 객체	goal 수락 여부, goal ID 등 포함
<code>get_result_async()</code>	Action 결과 수신	<code>GetResult.Response</code> 객체	최종 상태 + 사용자 정의된 결과 포함
<code>cancel_goal_async()</code>	goal을 취소하는 비동기 요청	<code>CancelGoal.Response</code>	취소 수락여부 및 취소된 ID

Service (client)

항목	설명
함수	<code>client.call_async(request)</code>
리턴값	<code>Future</code> 객체 (<code>rclpy.task.Future</code>)
<code>.result()</code> 타입	서비스 응답 메시지 (<code>Response</code> 타입 객체, 예: <code>srv.SomeService.Response</code>)

속성 / 메서드	설명
<code>done()</code>	<code>bool</code> - 응답이 완료되었는지 여부
<code>result()</code>	서비스 응답 객체 반환 (완료되었을 때만 유효)
<code>exception()</code>	예외가 발생했다면 해당 예외 객체 반환
<code>cancel()</code>	<code>Future</code> 를 취소 시도
<code>cancelled()</code>	<code>bool</code> - 취소되었는지 여부
<code>add_done_callback(fn)</code>	완료 시 호출될 콜백 함수 등록

Service (server)

request와 response를 argument로 받음

Action

항목	설명
함수	<code>send_goal_async(goal_msg)</code>
리턴값	<code>Future</code> 객체
<code>.result()</code> 타입	<code>rclpy.action.client.ClientGoalHandle</code> 객체

속성 / 메서드	설명
----------	----

<code>accepted</code>	<code>bool</code> 값 – 서버가 goal을 수락했는지 여부
<code>goal_id</code>	고유 goal ID (<code>UUID</code>)
<code>get_result_async()</code>	결과 요청용 비동기 함수
<code>cancel_goal_async()</code>	goal을 취소하는 비동기 요청
<code>goal</code>	클라이언트가 보낸 goal 메시지 자체

항목	설명
함수	<code>goal_handle.get_result_async()</code>
리턴값	<code>Future</code> 객체
<code>.result()</code> 의 타입	<code>GetResult.Response</code> 객체
내부 필드	<code>.status</code> , <code>.result</code> (→ 실제 결과 메시지)

필드	설명
<code>status</code>	Goal 상태 코드 (<code>int</code>): 예) <code>2 = SUCCEEDED</code> , <code>5 = ABORTED</code> 등
<code>result</code>	사용자 정의 Action의 결과 메시지 (예: <code>Fibonacci.Result.sequence</code>)

상수	의미
<code>GoalStatus.STATUS_ACCEPTED</code>	0
<code>GoalStatus.STATUS_EXECUTING</code>	1
<code>GoalStatus.STATUS_SUCCEEDED</code>	2
<code>GoalStatus.STATUS_CANCELED</code>	3
<code>GoalStatus.STATUS_ABORTED</code>	5

항목	설명
함수	<code>goal_handle.cancel_goal_async()</code>
리턴값	<code>Future</code> 객체
<code>.result()</code> 의 타입	<code>CancelGoal.Response</code> 객체 (<code>action_msgs.srv.CancelGoal.Response</code>)
내부 필드	<code>return_code</code> , <code>goals_canceling</code>

필드명	설명
<code>return_code</code>	요청 처리 결과 코드 (<code>int</code>) — 성공, 실패 등
<code>goals_canceling</code>	취소가 시도된 goal ID 목록 (<code>UUID</code> 리스트)

상수 (from <code>CancelGoal</code>)	의미
<code>CancelGoal.ERROR_NONE</code> (0)	성공적으로 취소 요청
<code>CancelGoal.ERROR_REJECTED</code> (1)	취소 거부
<code>CancelGoal.ERROR_UNKNOWN_GOAL_ID</code> (2)	알 수 없는 goal ID
<code>CancelGoal.ERROR_GOAL_TERMINATED</code> (3)	이미 완료/중단된 goal

cf . Underscore naming rules in Python

형식	공식 명칭 (한/영)	의미	예시	설명
<code>_var</code>	"프로텍티드 변수 관례 (Protected Variable Convention)"	내부용, 외부에서 직접 사용하지 말라는 의미	<code>_hidden_value</code>	실제 접근 가능하나, 관례상 내부용으로 간주
<code>__var</code>	"이름 뱀글링 (Name Mangling)"	클래스 내부 전용 (외부 접근 방지)	<code>__secret</code> → <code>__ClassName__secret</code>	하위 클래스와의 이름 충돌 방지 목적
<code>var_</code>	"후행 언더스코어 (Trailing Underscore)"	파이썬 키워드와의 충돌 회피	<code>class_</code> , <code>type_</code> , <code>id_</code>	<code>class</code> 등 예약어를 변수명으로 쓰고 싶을 때
<code>__var__</code>	"매직 메서드 또는 던더 메서드 (Magic Method or Dunder Method)"	파이썬이 특별하게 해석하는 예약 메서드	<code>__init__</code> , <code>__len__</code> , <code>__str__</code>	오버라이딩 가능. 직접 정의 시 특별한 의미를 가져야 함